# Class Activity: Hackathon 1 Preparation

Project 01B: Finding the Next Best Step

Team DSIders

This submission continues from our Solution Architecture activity. That report defined the inputs and outputs, a four-layer architecture (client, API, engine and data) and the data each engine service needs. Here we turn that architecture into a plan for Hackathon 1: what the minimum working product is and how we will demonstrate it, the tasks needed to build it as GitHub issues mapped to the architecture diagram, and the synthetic data we will generate to build and test it.

## 1 Goal: MVP and End-to-End Demonstration

### 1.1 MVP definition

For Hackathon 1, our MVP is a working version of the engine, running on one laptop, with a minimal mobile web front end. It takes a student from a typed goal to one next step for three career roles, and it sends unclear cases to a caseworker's list. All data in the MVP is synthetic.

We chose three roles that appear in our design work and that differ enough to test the engine properly: **data analyst**, **web developer** and **UI/UX designer**.

| **In the MVP** |
|---|
| Goal typed as one English sentence |
| Step 1 parser turning the sentence into a profile, with unknowns marked |
| Resume upload |
| Step 2 asking one question at a time, with a reason, and stopping early |
| Full login and account management (two seeded demo accounts instead) |
| Step 3 grouping results and finding the nearest next step from the graph |
| Step 4 handing low-confidence cases to a NGO |
| Student screens and NGO worker screens |
| Print, notes, NGO worker profile and availability |
| Sessions, handovers and overrides saved in MongoDB |

### 1.2 How we will judge the MVP

We will report four numbers from the synthetic test set. The first comes from Sprint 1; the other targets are our own and may change once we see the baseline.

| **Metric** | **What it tells us** | **Target** |
|---|---|---|
| Average questions before the result settles | Whether the question flow is short enough | Under 5 |
| Precision of the *already have* group | Whether we wrongly tell students they have something | At least 90% |
| Blocked cases that still get a reachable step | Whether a gap becomes a task, not a dead end | At least 90% |
| Handover rate on cases labelled ambiguous, and on clear cases | Whether Step 4 hands over when it should, and not too often | At least 80% and at most 15% |

### 1.3 End-to-end demonstration plan

The demo follows one student through the whole system and then shows one case that goes to a person. Each step touches specific blocks of the architecture diagram, so that the whole system is seen working.

| # | What we show | Blocks used |
|---|---|---|
| 1 | Priya logs in with a demo account and types “I want to become a data analyst but I don’t know where to start”. | Student app, FastAPI |
| 2 | We show the profile JSON returned by Step 1, with the fields it could not find marked as unknown. | Step 1, MongoDB |
| 3 | Priya answers two or three questions. Each card shows why it was asked, and one answer is “I don’t know yet”. | Step 2, Student app |
| 4 | The app shows what she already has, what is missing and what can wait, and one next step: write a first resume. | Step 3, requirement graph |
| 5 | She taps “I’ve done this”, the engine runs again, and the next step changes to learning basic SQL. | Step 1–Step 3, MongoDB |
| 6 | A second persona, Sana, is unsure of her goal. Step 4 does not pass her result. If Sana is not satisfied with the response of the app, she can messgae the NGO worker by clicking on a button. | Student app |
| 7 | The NGO worker receives the message and can respond to it. | NGO worker app |
| 8 | We close with the four metrics from the evaluation script on the synthetic test set. | Evaluation, curated data |

**Backup plan.** If the language model is slow on the day, we switch S1 to cached outputs for the demo personas so the rest of the flow still runs live. We will also keep a short screen recording of a full run.

# 2 Implementation Plan

## 2.1 Task breakdown

We split the work into 19 GitHub issues. Every issue belongs to one block of the architecture diagram, except three cross-cutting issues for setup, evaluation and the demo. Issues marked P0 are needed for the demo, and P1 issues are stretch goals. We will build in four phases:

1. **Foundations:** repository, database and graph format (#1–#3).
2. **Data and engine:** role graphs, synthetic data and the four services, with Step 3 first because the others exist to feed it (#4–#12).
3. **API and apps:** the orchestration endpoint and the student and caseworker screens (#13–#17).
4. **Evaluation and demo:** metrics, demo personas and rehearsal (#18–#19).

## 2.2 Issues mapped to the architecture

Figure 1 is the architecture diagram from our previous activity, with the issue numbers for each block shown in orange. Table 1 lists every issue. The diagram shows the four layers of the MVP (client, API, engine and data). Bhashini and the WhatsApp link are no longer drawn, because neither is part of the Hackathon 1 MVP: a student contacts the NGO worker through a message button inside the app.

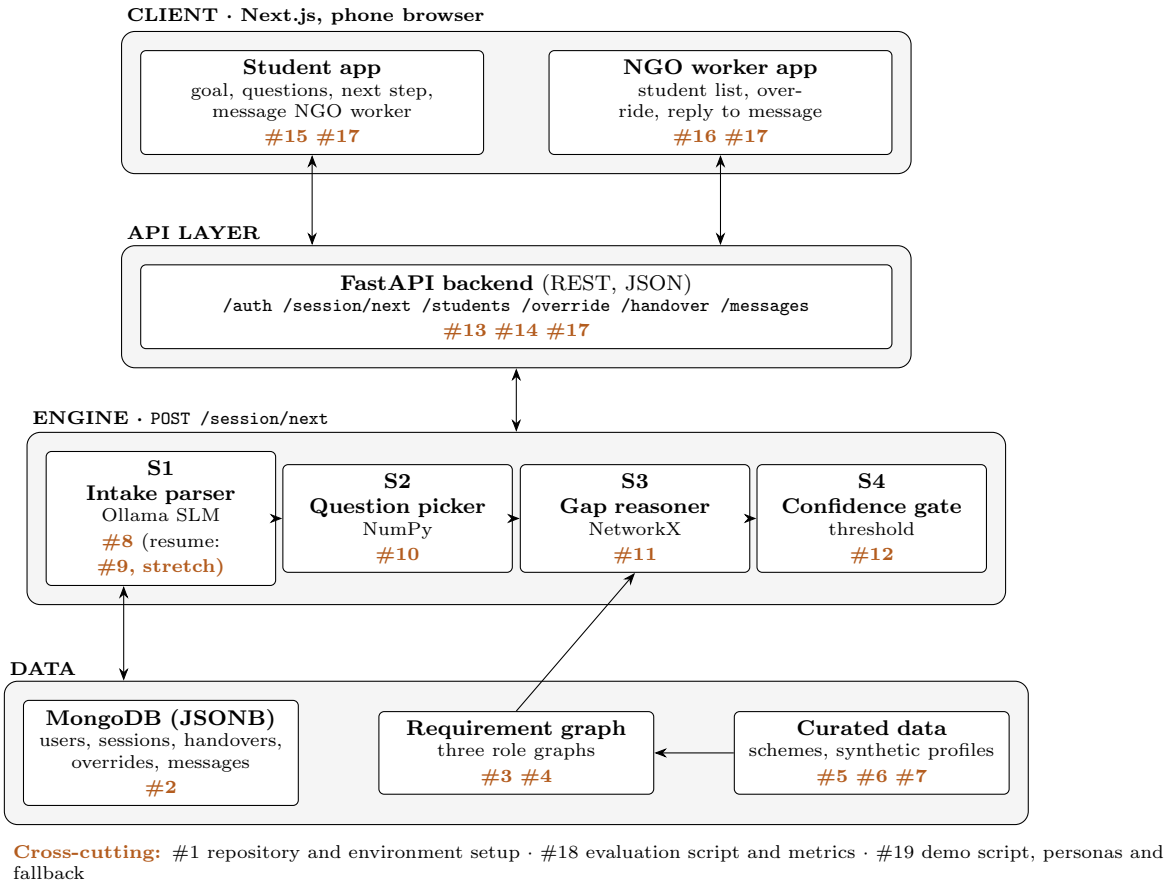



Figure 1: Architecture diagram with GitHub issue numbers for each block.

Table 1: GitHub issues for Hackathon 1.

| # | Issue | Architecture block | Pri. | Depends on |
|---|---|---|---|---|
| #1 | **Set up repository and local environment** Done when: Repo has client/, api/, engine/, data/ folders; Docker Compose starts MongoDB and Ollama; README explains how to run it on one laptop. | Cross-cutting (all blocks) | P0 | – |
| #2 | **Design MongoDB schema and seed script** Done when: Tables for users, profiles, sessions, answers, handovers, overrides and messages exist; seed script loads the two demo accounts. | Data: MongoDB | P0 | 1 |
| #3 | **Define requirement-graph format and loader** Done when: Node and edge types are documented; a NetworkX loader reads the JSON files and rejects cycles. | Data: Requirement graph | P0 | 1 |
| #4 | **Write requirement graphs for the three roles** Done when: Data analyst, web developer and UI/UX designer graphs pass the loader; every node has time, urgency and where to get help. | Data: Requirement graph | P0 | 3 |
| #5 | **Collect 10 to 15 scheme and course rules** Done when: Rules from myScheme and OGD are stored as JSON with source links and linked into the role graphs. | Data: Curated data | P1 | 3 |
| #6 | **Build the synthetic profile generator** Done when: Script creates complete profiles, gold labels and incomplete variants from a fixed seed, and writes JSONL files. | Data: Curated data | P0 | 3, 4 |
| #7 | **Create evaluation sets and goal sentences** Done when: English goal sentences, ambiguous cases and a hand-checked parser test set exist; dev and test splits are fixed. | Data: Curated data | P0 | 6 |

| # | Issue | Architecture block | Pri. | Depends on |
|---|---|---|---|---|
| #8 | **S1 intake parser: goal sentence to profile JSON**<br>Done when: Ollama model returns valid JSON for a typed goal; fields not stated are marked unknown; accuracy is measured on the parser test set. | Engine: S1 | P0 | 1 |
| #9 | **S1 resume parsing (stretch)**<br>Done when: A text PDF or photographed resume gives the same profile fields as typed text; fails gracefully on bad files. | Engine: S1 | P1 | 8 |
| #10 | **S2 question picker**<br>Done when: Given a partial profile, returns one question with a reason; stops early when the gain is low; never repeats a question. | Engine: S2 | P0 | 4, 8 |
| #11 | **S3 gap reasoner**<br>Done when: Returns have, missing and later groups with reasons, plus the nearest next step; matches gold labels on complete profiles. | Engine: S3 | P0 | 4 |
| #12 | **S4 confidence gate and handover**<br>Done when: Threshold is set in config; low confidence or "I don't know" creates a handover; every decision is logged. | Engine: S4 | P0 | 11 |
| #13 | **API: POST /session/next**<br>Done when: Runs S1 and S2, then S3 and S4; saves the session and returns groups, next step and pass or handover; re-runs after "I've done this". | API: FastAPI | P0 | 2, 8, 10, 11, 12 |
| #14 | **API: login, students, handover, override and messages**<br>Done when: Two demo accounts can log in; NGO worker can list students, open one and save an override; both sides can send and read messages. | API: FastAPI | P0 | 2, 13 |
| #15 | **Student app screens**<br>Done when: Figma frames 1 to 6 work in a phone browser against the live API, including "I don't know yet" and "I've done this". | Client: Student app | P0 | 13 |
| #16 | **NGO worker app screens**<br>Done when: Frames 9 and 10 work against the live API; new handovers appear at the top of the list. | Client: NGO worker app | P0 | 14 |
| #17 | **In-app messaging**<br>Done when: Student can message the NGO worker from the app; the worker sees it and can reply; messages are saved in MongoDB. | Client: both apps | P0 | 14, 15, 16 |
| #18 | **Evaluation script and metrics**<br>Done when: One command runs all test sessions with a simulated student, prints the four metrics and saves them to a dated file. | Cross-cutting (engine + data) | P0 | 7, 13 |
| #19 | **Demo script, personas and fallback**<br>Done when: Demo script written; five personas seeded; cached S1 output switch and screen recording ready; full run rehearsed twice. | Cross-cutting (all blocks) | P0 | 15, 16, 17, 18 |

## 2.3 GitHub setup

We will set up the repository so that the issue board mirrors the diagram.

- **Labels** name the architecture block of each issue: `layer:client`, `layer:api`, `engine:s1` to `engine:s4` and `layer:data`. Further labels mark `infra`, `synthetic-data`, `evaluation`, `demo`, and priority `P0` or `P1`.
- **Milestone:** all 19 issues sit under a single *Hackathon 1* milestone, so progress is visible in one place.
- **Project board** with four columns: To do, In progress, In review and Done.
- **Workflow:** one branch per issue, and a pull request that closes the issue and is reviewed by one other team member before merging.

We have written a short script, `create_github_issues.sh`, that creates these labels, the milestone and all 19 issues using the GitHub command-line tool, so the board matches this document exactly. Each issue body records its architecture block, its "done when" condition and the issues it depends on.

# 3 Synthetic Data Follow-up

## 3.1 Expanding on the last class task

In the last activity we listed the parameters and outputs of each engine service. To generate data, we now need to fix exactly what each record contains. We will create four kinds of data.

**1. Student profiles.** One record per synthetic student. Every field also stores a confidence and a source (stated, asked or inferred), because S4 depends on these.

| Field | Type | Values used in generation |
|---|---|---|
| `goal_text` | text | The student's own English sentence (see step 4 below) |
| `target_role` | category | data analyst, web developer, UI/UX designer, not sure |
| `degree`, `year` | category | B.Sc, B.Com, B.A., B.Tech, diploma; second or final year |
| `district` | category | Hyderabad, Rangareddy, Medchal, Warangal, Karimnagar and other Telangana districts |
| `skills` | list | For example Excel, SQL, HTML, Python, Canva, Figma, with a level from none to basic to good |
| `has_resume` | category | ready, started, no, not sure |
| `projects` | number | 0 to 3 |
| `documents` | list | marksheets, ID, certificates available or missing |
| `language` | category | English in the MVP; Telugu and Hindi are kept in the schema for later |

**2. Requirement graphs.** One graph per role. Nodes are roles, skills, artefacts (such as a resume or portfolio), documents, courses and schemes. Edges say *requires* or *helps with*. Every node holds a time estimate, a cost (usually free), where to get help, and whether it is needed now or later.

**3. Demo accounts.** Two seeded accounts replace full login and account management: one student and one NGO worker. The NGO worker record holds only a name and a centre, and is used to show the student who they are messaging. NGO worker profiles, availability and matching are outside the MVP.

**4. Labels for evaluation.** For each test session, the correct *have*, *missing* and *later* groups, the correct next step, and a flag that marks the case as clear or ambiguous.

## 3.2 Plan for generating and processing the data

The idea is simple. We first create *complete* students whose correct answers we know, then hide parts of each one, and finally check whether the engine can recover what we hid.

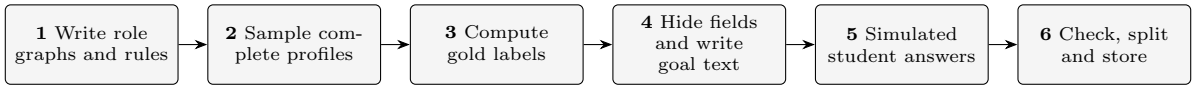



Figure 2: Synthetic data pipeline.

1. **Write the role graphs and rules by hand.** We will use public job descriptions and free course syllabi to decide what each role needs, and take a small set of scheme and course rules from myScheme and the Open Government Data platform. This is the only manual step, and it is the part we most want Barabari NGO workers to check.
2. **Sample complete profiles.** A Python script draws each field from simple distributions that match our users (mostly final-year students from Telangana colleges, with few having a resume). A fixed random seed makes every run reproducible. We also add rules so that profiles make sense together, for example a student who lists SQL as "good" usually also has Excel.

3. **Compute gold labels.** For each complete profile, we walk the role graph to work out the correct groups and the correct next step. Because this uses the full profile, the answer is known exactly.
4. **Hide fields and write goal text.** We make three versions of each profile: one with about 30% of fields removed, one with about 50% removed, and one with about 70% removed. For each version we write a one-sentence English goal from varied templates, then use the local language model to paraphrase it into more natural and colloquial wording, including some spelling mistakes. We record exactly which fields were hidden.
5. **Simulate the student's answers.** During testing, a small "answer oracle" replies to S2's questions, one at a time, using the hidden complete profile. To make it realistic, it sometimes answers "I don't know yet" and sometimes gives a slightly wrong answer.
6. **Make ambiguous cases on purpose.** Some profiles get conflicting information, such as claiming good SQL with no other data skills, or a goal that names two roles. These are labelled ambiguous and should be handed over by S4 to the NGO worker.
7. **Check, split and store.** Every record is validated against a JSON schema, and duplicates are removed. Then a team member reads a random 10% to check that it looks believable. The data is split into a development set (for tuning S2 and S4 thresholds) and a test set (for the reported numbers). It is stored as JSONL files in the repository, and a seed script loads it into MongoDB.

## 3.3 Quantity of data for Hackathon 1

The engine is mostly rule-based, so this data is for testing and tuning rather than for training a model. The amounts below are large enough to give stable numbers, but small enough to generate on a laptop and to check a sample by hand.

| **Data** | **Amount** | **Reasoning** |
|---|---|---|
| Role requirement graphs | 3 graphs, about 25–40 nodes each | One per MVP role; small enough to write and check by hand |
| Scheme and course rules | 10–15 | Only those linked to the three roles |
| Complete seed profiles | 330 | 100 per role, plus 30 students who are not sure of their goal |
| Incomplete test sessions | about 1,000 | 330 profiles at three hiding levels |
| of which ambiguous | about 60 | Enough to measure the S4 handover rate on ambiguous cases (target at least 80%) and on clear cases (target at most 15%) |
| Development / test split | 300 / 700 | 700 test sessions give each reported percentage a margin of about ±4 points |
| Parser test sentences | 150, hand-checked | To measure S1 field accuracy directly |
| Synthetic resumes (stretch) | 60 PDFs and 20 photos | Only if issue #9 is reached; resume upload is not in the MVP |
| Demo personas | 5 | Priya, Sana, Arjun, Divya and Mahesh, matching the Figma screens; Priya is the main demo and Sana is the unsure student handed to the NGO worker |
| Demo accounts | 2 | One student and one NGO worker, seeded instead of full login |

All of this is under a few megabytes in total. None of it is real student data, so the results will be a benchmark of the engine and not proof that it works for real students. After Hackathon 1, we would like to compare it against anonymised case notes from our partner organisation.